PROJECT GUIDE | LOCAL-FIRST AI

Secure Enterprise RAG Workspace

A practical guide to the problem, architecture, technology choices, security model, benefits, limits, and use cases.

PRIVATE DOCUMENTS	LOCAL AI PIPELINE
Upload and index files Ask questions in natural language See the passages behind each answer	React + FastAPI PostgreSQL + Qdrant SentenceTransformers + Ollama

What this project demonstrates

A local document question-answering system that retrieves relevant passages from uploaded files and uses a local language model to produce a response grounded in those passages. It includes accounts, role-based permissions, document history, feedback, tags, and controlled document sharing.

Prepared for explaining the working prototype to project coordinators

1. The problem and the project idea

Organizations accumulate policies, certificates, offer letters, manuals, and reports. Finding one precise fact can mean opening files one by one, searching inconsistent wording, and repeating the same work. General-purpose chatbots may also answer from outside knowledge, which makes it hard to verify where an answer came from.

PROBLEM

Information is buried in many documents. Keyword search misses paraphrases, and unsupported AI answers are difficult to trust.

SOLUTION

Retrieve semantically relevant passages from permitted files, then ask a local language model to answer from those passages and show source excerpts.

Problem statement

Design and demonstrate a local-first, access-controlled document Q&A; application that lets an authenticated user upload supported business documents, search their contents with natural-language questions, and inspect the evidence returned with each answer.

What the system does

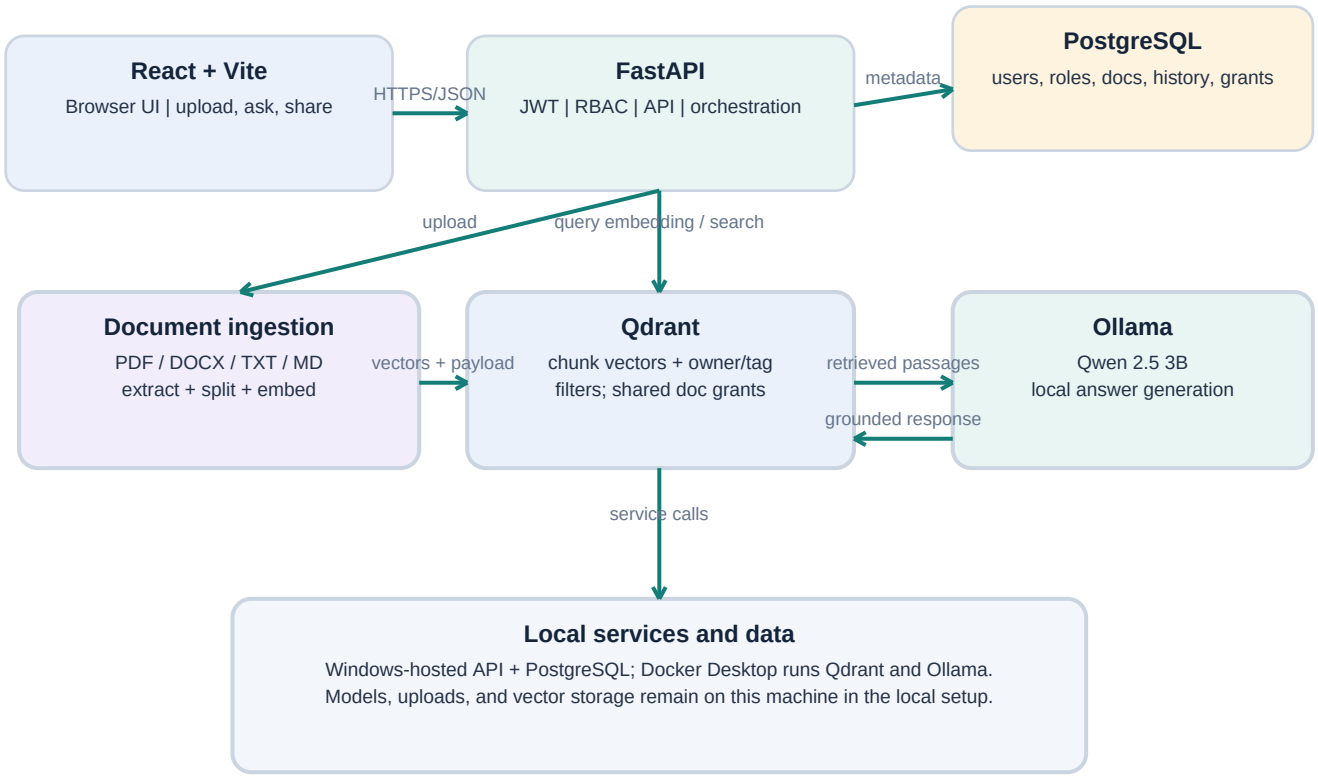
- Converts PDF, DOCX, TXT, and Markdown files into searchable text chunks.
- Finds semantically similar passages even when a question uses different wording.
- Uses the retrieved passages as context for a local language model and displays source names and excerpts.
- Keeps document search scoped to the owner, administrator access rules, tags, and explicitly granted shares.
- Saves recent questions, supports answer feedback, and offers administrator account/history tools.

Intended outcome

A working prototype and demonstrable architecture for private, source-grounded knowledge access. It reduces manual searching, while leaving people responsible for checking important answers against the cited source.

2. System architecture

The browser handles the workspace experience. FastAPI authenticates requests, checks permissions, and coordinates ingestion and question answering. PostgreSQL stores account and document records. Qdrant stores searchable chunk vectors. Ollama runs the local answer model.



The diagram describes the current local development deployment, not a high-availability production cluster.

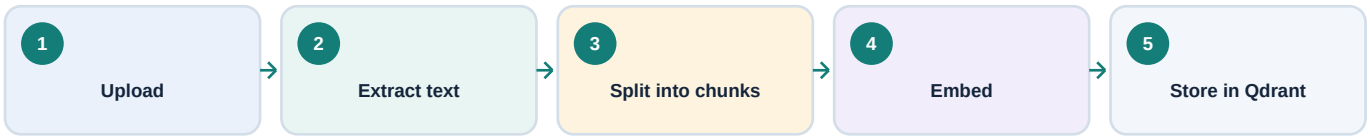
Data stays on the Windows machine in the documented local setup. PostgreSQL runs as the existing Windows service; Docker Compose starts Qdrant and Ollama. The API and Vite development server run as local processes. An alternative embedded Qdrant mode is configurable.

Main data stores

- PostgreSQL: users, roles, document metadata, tags, query history, feedback, login attempts, and share/grant records.
- Qdrant: chunk embeddings and payload fields such as owner ID, document ID, filename, chunk number, text, and tags.
- Local upload folder: original files, organized under the backend data directory by owner.
- Ollama model storage: downloaded model files held in the Docker volume.

3. How a question becomes an answer

Document ingestion happens when a user uploads a supported file. The system extracts text, breaks it into overlapping chunks, converts each chunk into a numeric embedding, and stores the text and vector together with access-related metadata.



Question time: embed the question -> retrieve allowed passages -> generate with Ollama -> show answer and sources

Question answering

- The signed-in user asks a question, optionally narrowing search by tag.
- SentenceTransformers converts the question into an embedding using the same embedding space as the stored chunks.
- Qdrant finds up to five similar chunks and applies owner/admin and granted-document filters.
- FastAPI sends the question and retrieved passages to Ollama with instructions to answer from that context and say when evidence is missing.
- The interface displays the answer, source filenames, chunk numbers, and excerpts. The question and answer can be saved to history and rated.

Why retrieval matters

The language model does not need to memorize each uploaded file. Retrieval selects a small set of relevant evidence at question time. This makes answers easier to trace and allows files to be added without retraining the model. Retrieval quality still depends on readable source text, useful chunking, and the wording of the question.

A key distinction

RAG means retrieval-augmented generation: retrieval supplies document evidence; generation turns that evidence into a readable response. The citations shown in this prototype identify retrieved files and chunks. They are helpful verification aids, not a formal guarantee that every sentence is fully supported.

4. Technology choices and why they are used

Technology	Role in the project	Why it fits this prototype
React + Vite	Browser workspace and development server	Quick interactive UI for upload, questions, document lists, history, feedback, and sharing.
FastAPI + Python	REST API, auth, ingestion and RAG orchestration	Python ecosystem integrates naturally with ML, parsing, and vector search libraries; automatic API docs aid inspection.
PostgreSQL + SQLAlchemy	Relational records and data relationships	Stores users, roles, documents, history, feedback, and sharing records with constraints and transactions.
Qdrant	Vector search for chunk embeddings	Purpose-built vector database with payload filters for owners, tags, and granted document IDs.
SentenceTransformers / all-MiniLM-L6-v2	Text and question embeddings (384 dimensions)	Compact sentence embedding model for semantic similarity; configured to run on CPU locally.
Ollama / Qwen 2.5 3B	Local language-model answer generation	Runs the selected model on the local machine without sending prompts to a hosted LLM API.
LlamaIndex	Sentence-aware chunk splitting and model integrations	Provides convenient document-processing and embedding/LLM integrations without requiring a hosted platform.
Docker Compose	Starts Qdrant and Ollama services	Repeatable local service setup and persistent volumes for vector/model data.
JWT + RBAC	Signed-in identity and role checks	API can distinguish an authenticated user from an administrator and apply permissions to requests.

Operational note

The embedding model downloads from Hugging Face on first use, then is cached locally. The Ollama model is pulled into a Docker-managed volume once. This makes normal inference local after the initial setup; it does not mean the first setup requires no network.

5. Privacy, permissions, and controlled sharing

Access is enforced in the API and vector search. The application is designed so that a normal user searches their own documents plus documents specifically shared with their account. Administrators have broader workspace access.

CONTROL	HOW IT WORKS HERE
Authentication	Users sign in and receive a JWT access token. Protected API calls identify the account from that token.
Role checks	Standard users and administrators have different permissions; admin account management and history viewing are separate routes.
Document isolation	Qdrant search filters by authenticated owner ID and any document IDs granted to that user. Admin search can span the workspace.
One-time share key	Owner or admin creates a random key; only its SHA-256 hash is stored. It expires after seven days if unused.
Redeemed share	A signed-in recipient redeems the key once. They can then read/search that shared document until the owner or admin revokes access. Recipients cannot delete or re-share it.
Login throttling	Five failed attempts for an account/device key within 15 minutes temporarily rate-limit further attempts.
Data-conscious logs	Request IDs, endpoint templates, status, and timings help diagnostics; README says logs omit document/question contents and credentials.

Privacy boundary

The configured local deployment avoids sending document text and prompts to a hosted language-model API. Network access may still be used to download models, install packages, or obtain software updates. Local processing is a deployment choice, not a blanket security certification: machine access, Windows accounts, backups, network exposure, secrets, and Docker configuration still matter.

6. Benefits and trade-offs

ADVANTAGES

Private by deployment: local model serving and local files.

Traceable: source names, chunks, and excerpts accompany the response.

Useful search: semantic retrieval can match related wording.

Permission-aware: user ownership, admin role, tags, and explicit share grants shape access.

Extensible: modular API, database, vector store, and UI support incremental improvements.

LIMITATIONS

Answer quality: a small local model can misunderstand, omit details, or invent unsupported content.

Hardware: first model downloads take space; CPU inference can be slow. GPU setup is not configured by default.

Input limits: uploads are capped at 20 MB; scanned/image-only PDFs need OCR, which is not part of current extraction.

Prototype scope: local single-machine setup does not provide high availability, disaster recovery, centralized identity, or production operations.

Security work remains: production use needs deployment hardening, HTTPS, secret handling, backup policy, monitoring, and security review.

What to say about reliability

The application is designed to ground answers in retrieved passages and to state when no relevant material is found. A good demo should still check the cited excerpt, ask a question whose answer is absent, and confirm the system responds that it cannot find it. Document Q&A; is decision support, not a substitute for reviewing authoritative records.

Possible next improvements

- OCR for scanned PDFs; show a clear message when extraction finds no readable text.
- Add document-level audit events and admin reports for uploads, shares, and revocations.
- Evaluate retrieval and answer quality on a curated set of question/answer/source examples.
- Add backup/restore procedures, HTTPS deployment, stronger secret management, and production configuration before any organizational rollout.

7. Applications and coordinator demo

The same workflow can support teams who need fast answers from a bounded collection of documents, provided each user is authorized to access those files.

HR and onboarding

Find dates, benefits, requirements, and policies in offer letters or employee guides.

Education and training

Ask about course certificates, syllabi, training materials, or project documentation.

Operations

Search procedures, checklists, and internal manuals.

Compliance support

Locate clauses and evidence in policy collections for human review.

Research and project work

Gather supporting passages across reports and notes.

Suggested 5-minute demonstration

1	Sign in as a normal user and upload a text-based certificate or offer letter.
2	Ask one question whose answer is plainly stated; open the cited source excerpt.
3	Ask for information not in the document, such as a salary when none is stated; observe the uncertainty response.
4	Sign in with a second account; verify the first account’s private file is not in its search results.
5	Create a one-time share key, redeem it in the second account, confirm read-only access, then revoke it.

Coordinator takeaway

This project demonstrates an end-to-end, locally run RAG application: document ingestion, semantic retrieval, local generation, source display, and permission-aware access. Its value is a practical, explainable prototype; organizational production use would require further operational and security engineering.

8. Quick glossary and project references

RAG	Retrieval-Augmented Generation: find relevant source text first, then use it to help generate an answer.
Embedding	A list of numbers representing semantic meaning, used to compare the similarity of questions and text chunks.
Vector database	A store that indexes embeddings and retrieves nearby matches; here, Qdrant.
Chunk	A smaller piece of a document indexed and retrieved independently.
JWT	A signed token used to carry authenticated identity between a client and API.
RBAC	Role-Based Access Control: permissions depend in part on a user role such as user or admin.

Project references

The implementation details summarized here were checked against the project README, Docker Compose configuration, FastAPI routes and services, database models, application settings, and React interface. The README is the starting point for local setup. FastAPI interactive API documentation is available at **http://127.0.0.1:8000/docs** while the backend is running. The website runs at **http://127.0.0.1:5173** in development.

How to explain it in one sentence

“Our project lets authorized users ask questions about their own or explicitly shared documents; it retrieves matching passages, uses a local language model to write a grounded answer, and shows the evidence so people can verify it.”

Prepared as a learning and presentation aid. Product behavior and deployment configuration can change as the project evolves.